Trabalho Prático: Codificação de Linha Manchester em VHDL (Vivado)

Comunicação de Dados - Sistemas Reconfiguráveis

Objetivo

Desenvolver, simular e **implementar em FPGA**, utilizando o Vivado e a linguagem VHDL, codificadores de linha do tipo **Manchester** e **Manchester Diferencial**. O trabalho contempla a análise do comportamento dos sinais por meio de simulação no Vivado Simulator e a execução na placa física (Basys 3), com os *switches* representando a sequência de bits de entrada e os LEDs apresentando o sinal codificado na saída.

Diferentemente dos códigos NRZ, nos códigos Manchester o intervalo de bit é dividido em **duas metades** e sempre existe uma **transição no meio do bit**. Essa transição carrega a informação de sincronismo (o sinal é *autossincronizante*), ao custo de exigir aproximadamente o dobro da largura de banda de um código NRZ com a mesma taxa de bits.

Descrição da Atividade

Cada grupo deverá desenvolver **dois projetos independentes no Vivado**, sendo:

Projeto 1: Codificação Manchester

O projeto deverá conter:

- Um arquivo VHDL (**source**) com a implementação do codificador Manchester;
- Um arquivo VHDL (**testbench**) para simulação;
- Arquivo de **constraints** (`.xdc`) com o mapeamento dos pinos da placa.

Representação do sinal:

Na codificação Manchester, o bit é representado pelo **sentido da transição no meio do intervalo de bit**. Neste trabalho será adotada a convenção do padrão **IEEE 802.3** (Ethernet):

Bit	1ª metade	2ª metade	Transição no meio
0	nível alto	nível baixo	descida (alto → baixo)
1	nível baixo	nível alto	subida (baixo → alto)

Observação: existe também a convenção de G. E. Thomas, com os sentidos invertidos. Para padronizar a avaliação, **todos os grupos devem usar a convenção IEEE 802.3** e indicá-la explicitamente na documentação.

Projeto 2: Codificação Manchester Diferencial

O projeto deverá conter:

- Um arquivo VHDL (**source**) com a implementação do codificador Manchester Diferencial;
- Um arquivo VHDL (**testbench**) para simulação;
- Arquivo de **constraints** (`.xdc`) com o mapeamento dos pinos da placa.

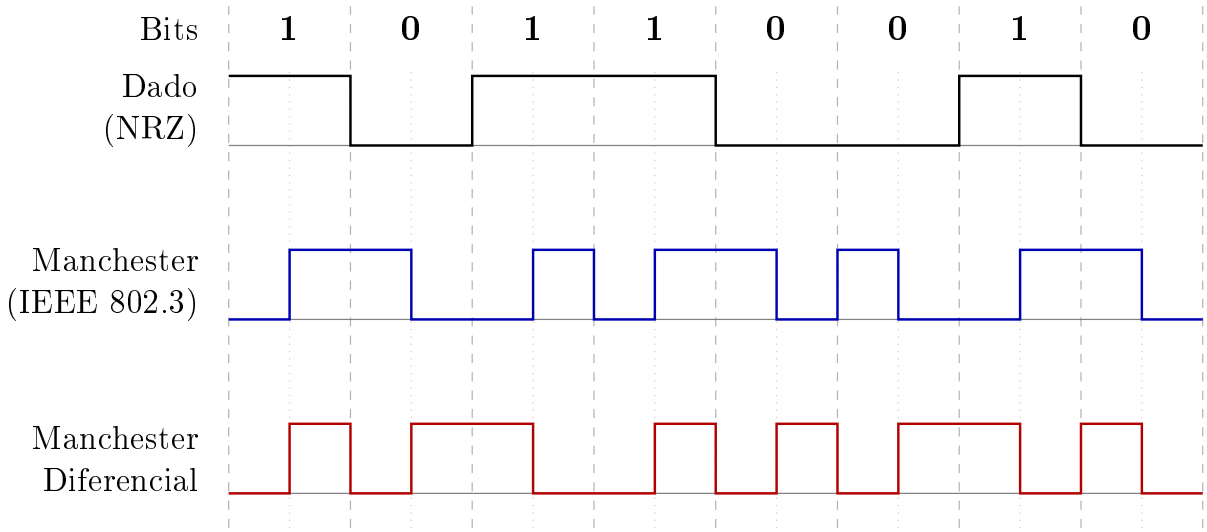
Representação do sinal:
Na codificação Manchester Diferencial, **sempre há uma transição no meio do bit** (usada apenas para sincronismo), e o valor do bit é representado pela **presença ou ausência de transição no início do intervalo de bit**:

Bit	Início do intervalo	Meio do intervalo
0	ocorre inversão do nível	ocorre inversão do nível
1	mantém o nível anterior	ocorre inversão do nível

Como a regra depende do nível anterior, o circuito precisa **memorizar o último nível transmitido**. O nível da saída após o *reset* deve ser **baixo**, e essa condição inicial deve ser respeitada no testbench e documentada.

Exemplo de formas de onda

A figura abaixo mostra a sequência `10110010` representada como dado NRZ (referência), em Manchester (IEEE 802.3) e em Manchester Diferencial (nível inicial baixo). As linhas tracejadas marcam o início de cada bit e as pontilhadas marcam o meio do bit.



Entrada de Dados

- A entrada de bits deverá ser um vetor com a **mesma quantidade de switches da FPGA** (16 bits na Basys 3, `SW15` a `SW0`).
- Cada *switch* da placa representa um bit da sequência a ser codificada (*switch* para cima = 1).
- A transmissão começa pelo bit mais significativo: primeiro `SW15`, depois `SW14`, e assim por diante até `SW0`.
- Após o último bit (`SW0`), a transmissão recomeça automaticamente em `SW15`, de forma cíclica.
- O botão central (`BTNC`) funciona como *reset*: enquanto pressionado, a transmissão fica parada e a saída em nível baixo; ao ser solto, a transmissão começa em `SW15`.
- O valor de cada *switch* deve ser lido no **início do intervalo do bit correspondente**, de modo que alterar um *switch* afete a sequência a partir da próxima vez que aquele bit for transmitido.
- No testbench, essa sequência deverá ser aplicada ao longo do tempo de forma equivalente à entrada física.

Saídas para observação do sinal

Saída pelos LEDs

Nos dois projetos, o sinal codificado deve ser apresentado no **LED que fica acima do *switch* cujo bit está sendo transmitido**. Todos os outros LEDs permanecem apagados. Dessa forma:

- A **posição** do LED ativo indica qual bit da sequência está sendo transmitido. Como a transmissão vai de `SW15` a `SW0`, o LED ativo percorre a fileira da esquerda para a direita, de `LD15` até `LD0`, e depois recomeça;
- O **valor do bit** é a posição do próprio *switch* logo abaixo do LED ativo;
- Dentro de cada posição, o LED apresenta as **duas metades do bit**: fica aceso na metade em que o sinal codificado está em nível alto e apagado na metade em que está em nível baixo.

Como nos dois códigos todo bit tem uma metade em nível alto e outra em nível baixo, o LED de cada posição sempre acende em algum momento, o que permite acompanhar a sequência inteira. A tabela abaixo mostra o que se observa em Manchester para `SW15` = 1 e `SW14` = 0:

Momento	LD15	LD14	LD13 a LD0
Bit de SW15 (1), 1ª metade	apagado	apagado	apagados
Bit de SW15 (1), 2ª metade	**aceso**	apagado	apagados
Bit de SW14 (0), 1ª metade	apagado	**aceso**	apagados
Bit de SW14 (0), 2ª metade	apagado	apagado	apagados

Como conferir na placa:

- **Manchester:** se o *switch* abaixo do LED ativo está em 1, o LED fica apagado e depois aceso; se está em 0, fica aceso e depois apagado;
- **Manchester Diferencial:** o LED ativo sempre muda de estado no meio do bit. Na passagem para a posição seguinte, o novo LED começa no **mesmo estado** em que o LED anterior terminou se o novo bit for 1, e no **estado oposto** se o novo bit for 0.

Pinos: o clock de 100 MHz da placa entra pelo pino `W5` e o botão `BTNC` pelo pino `U18`. Os pinos dos *switches* e dos LEDs estão no arquivo `Basys-3-Master.xdc`, disponibilizado pela Digilent. Recomenda-se partir desse arquivo, descomentando apenas as linhas utilizadas e renomeando as portas para os nomes usados no VHDL.

Sugestão: o display de 7 segmentos da placa pode mostrar o número do bit em transmissão (15 a 0), o que ajuda a acompanhar a sequência e a gravar o vídeo de demonstração.

Saída pelo conector PMOD (Osciloscópio)

Nos dois projetos, disponibilizar o sinal codificado em um pino do conector **PMOD** da placa, de forma que possa ser observado em um **osciloscópio** físico.

- Adicionar a saída do sinal codificado a um pino do PMOD no arquivo de *constraints*;
- Disponibilizar também o dado original em outro pino do PMOD, para observação simultânea nos dois canais do osciloscópio;
- Capturar a forma de onda no osciloscópio para os dois códigos (Manchester e Manchester Diferencial);
- Apresentar as capturas no relatório, identificando claramente os sinais e as transições no meio do bit.

Sugestão: para a observação no osciloscópio, utilizar um `CYCLES_PER_BIT` bem menor que o usado nos LEDs, de forma a obter uma taxa de bits na faixa de kHz.

Saída VGA com cores pulsantes

Nos dois projetos, utilizar a **saída VGA** da placa para apresentar visualmente o sinal codificado, alterando a cor de tela cheia conforme o nível do sinal:

- Sinal em **nível baixo** (0): tela em uma cor (ex: preto ou azul);
- Sinal em **nível alto** (1): tela em outra cor (ex: branco ou verde);

- A tela deve "piscar" conforme a sequência codificada é transmitida, evidenciando as duas metades de cada bit.

Sugestão: implementar o controlador VGA padrão (640x480 @ 60Hz) e gerar os sinais RGB em função do nível atual do sinal codificado.

Duração do Bit

A duração de cada bit (intervalo de símbolo) deverá ser **parametrizável** tanto no circuito principal quanto no testbench. Como o sinal muda de nível no meio do bit, o circuito precisa controlar também o **instante correspondente à metade do intervalo**.

Circuito Principal

- A duração do bit deve ser definida em função do **número de ciclos de clock**;
- Recomenda-se o uso de **generics** ou constantes (ex: `CYCLES_PER_BIT`);
- O valor de `CYCLES_PER_BIT` deve ser **par**, para que as duas metades do bit tenham exatamente a mesma duração (alternativamente, pode-se parametrizar diretamente `CYCLES_PER_HALF_BIT`).

A duração do bit deve ser definida em função do número de ciclos de clock, não por assumir periodicidade do sinal, mas por se tratar de um sistema digital síncrono, no qual o tempo é discretizado pelo clock.

Para a execução na placa, o valor da contagem deve ser dimensionado de modo que **as duas metades do bit sejam visualmente perceptíveis nos LEDs**. Recomenda-se de 1 a 2 segundos por bit (ou seja, de 0,5 a 1 segundo por metade). Lembre-se de que o clock da Basys 3 é de 100 MHz.

Testbench

- A duração do bit deve ser definida em unidade de tempo;
- Deve ser facilmente modificável (ex: constante `BIT_PERIOD`);
- O valor de `BIT_PERIOD` deve ser coerente com o período de clock simulado e com o *generic* usado na simulação (`BIT_PERIOD` = `CYCLES_PER_BIT` × `CLK_PERIOD`). Exemplo: clock de 10 ns e `CYCLES_PER_BIT` = 10 resultam em `BIT_PERIOD` = 100 ns;
- A sequência de bits deve respeitar esse intervalo;
- Para o Manchester Diferencial, a sequência de teste deve conter **bits 0 e 1 consecutivos e alternados**, de modo a evidenciar os dois casos da regra de transição no início do bit;
- Recomenda-se o uso de `assert` para verificar automaticamente o nível esperado em cada metade de bit.

Dicas de Implementação

Uma forma de organizar o circuito é dividi-lo nos blocos abaixo. A organização é apenas uma sugestão; o grupo pode adotar outra arquitetura, desde que a descreva na documentação.

1. **Contador de ciclos:** conta de 0 até `CYCLES_PER_BIT`−1 e volta a zero. O fim da contagem marca o fim do bit, e o valor `CYCLES_PER_BIT/2` marca o meio do bit. O sinal que indica em qual metade do bit o circuito está pode ser obtido diretamente da comparação do contador com esse valor.

2. **Índice do bit:** contador de 15 até 0 que decrementa a cada fim de bit e volta para 15 após o bit 0. Ele seleciona qual *switch* está sendo transmitido e qual LED recebe o sinal codificado.

3. **Registrador do bit atual:** no início de cada bit, guarda `data_in(indice)`. É esse registrador que alimenta o codificador, garantindo que o bit não mude no meio do intervalo se o *switch* for alterado.

4. **Codificador Manchester:** a saída depende apenas do bit atual e da metade do bit. Monte a tabela-verdade com essas duas entradas (são só quatro linhas) e observe qual porta lógica ela representa.

5. **Codificador Manchester Diferencial:** precisa de um registrador com o nível atual da saída. No início de cada bit, o registrador é invertido se o bit for 0 e mantido se for 1; no meio do bit, é sempre invertido. No *reset*, o registrador vai para nível baixo.

6. **Saída nos LEDs:** a saída dos LEDs é um vetor de 16 bits em que apenas a posição indicada pelo índice recebe o sinal codificado; as demais posições recebem `'0'`. Isso pode ser descrito com um `process` que zera o vetor e depois atribui o sinal codificado à posição do índice.

7. **Saídas registradas:** é recomendável que o sinal codificado saia de um *flip-flop* e não diretamente de lógica combinacional. Isso evita pequenos pulsos espúrios (*glitches*) que não aparecem nos LEDs, mas aparecem no osciloscópio.

8. **Botão:** o sinal do `BTNC` é assíncrono em relação ao clock. Recomenda-se passá-lo por dois *flip-flops* em série (sincronizador) antes de usá-lo como *reset*.

Dimensionamento: com clock de 100 MHz, 1 segundo por bit corresponde a `CYCLES_PER_BIT` = 100 000 000, valor que cabe em um contador de 27 bits ou em um `integer` com faixa `range 0 to CYCLES_PER_BIT-1`. No testbench, o mesmo componente deve ser instanciado com um valor pequeno de `CYCLES_PER_BIT` via `generic map`, sem alterar o código do circuito principal.

Simulação

Os alunos deverão utilizar o **Vivado Simulator** para:

- Executar a simulação comportamental;
- Visualizar as formas de onda, exibindo lado a lado o índice do bit, o bit atual, o sinal codificado e o vetor dos LEDs;
- Verificar a transição no meio de todos os bits e, no caso do Manchester Diferencial, a presença ou ausência de transição no início de cada bit;
- Validar o funcionamento do codificador antes da gravação na FPGA.

Implementação na FPGA

Após a validação da simulação, os grupos deverão realizar a **síntese, implementação e gravação do bitstream** na placa FPGA. O sistema gravado deverá:

- Ler a sequência de bits configurada pelos *switches*;
- Iniciar a transmissão a partir de um sinal de *start* ou *reset* mapeado em um botão (*push-button*) da placa;
- Apresentar o sinal codificado no LED acima do *switch* em transmissão, com os demais LEDs apagados;
- Permitir a alteração da sequência de entrada via *switches* sem necessidade de regravação.

Critério de Avaliação

A nota do trabalho é dividida igualmente entre a **implementação** e o **repositório Git**:

Implementação (50%)	
Manchester: simulação comportamental correta (testbench com verificação das transições)	10%
Manchester: funcionamento na placa (sinal codificado no LED da posição em transmissão, ordem de transmissão, *reset*, troca de sequência pelos *switches*)	10%
Manchester Diferencial: simulação comportamental correta (incluindo nível após o *reset* e os dois casos da regra de transição)	10%
Manchester Diferencial: funcionamento na placa	10%
Saída PMOD com capturas no osciloscópio dos dois códigos	5%
Saída VGA com cores pulsantes nos dois códigos	5%
Repositório Git com todos os arquivos (50%)	
READMEs (raiz, de cada projeto e de cada saída PMOD/VGA)	10%
Tutoriais (simulação, placa, PMOD e VGA) com prints e fotos	15%
Documentação técnica de cada projeto	10%
Relatório técnico final e vídeo de demonstração	15%
Total	**100%**

A implementação é verificada pelas simulações, pelo vídeo de demonstração e pelas capturas apresentadas no relatório. O repositório é avaliado pela presença e pela qualidade de cada arquivo, conforme descrito no Guia de Entrega e no Guia de Documentação.

Observações Importantes

- Cada codificação deve ser feita em **projeto separado**;
- Cada projeto deve conter **um source, um testbench e um arquivo de constraints**;
- A convenção do Manchester adotada é a **IEEE 802.3** (0 = descida, 1 = subida no meio do bit);
- No Manchester Diferencial, o nível da saída após o *reset* é **baixo**;
- A implementação na placa é **obrigatória** nesta etapa;
- O foco continua sendo a **análise dos sinais**: simulação, execução na placa e visualização no osciloscópio e no VGA.